



BEAUTY OF CLOUD 2.0

ROUND 1

PROBLEM STATEMENTS (V3)



University of Sri Jayewardenepura
IEEE Student Branch



IEEE
COMPUTER
SOCIETY

University of Sri Jayewardenepura
Student Branch Chapter

General Information

- Released: Aug 12
- Scenario lock-in: Aug 18
- Submission deadline: Sep 6, 11:59 PM
- Cloud platforms allowed: AWS, GCP, or Microsoft Azure — pick whichever fits your solution, and justify the choice. You may mix providers if you can explain why.
- Changes from v2: Added a 5th scenario focused on AI/ML agents, and de-localized every scenario — none are tied to a specific country anymore. Teams can set their own regional context if they want one, but it's not assumed or required.

How to Read These Problem Statements

- Each statement below describes a real-world problem — not a solution. No AWS/GCP/Azure services are named on purpose. Your job as a team is to research, choose, and justify your own architecture. A proposal that just names services without explaining why they fit this specific problem will score lower than one with fewer services but clearer reasoning.
- These are large, multi-faceted systems — you are not expected to design every corner of them in full depth. Pick the parts of the system you'll go deep on, and say so explicitly, rather than skimming everything shallowly.
- There is no single correct answer. Two teams solving the same statement with completely different architectures can both score well, if both are well-justified.

1. Scenario 1 - Disaster Early Warning System

Background

Many regions around the world face recurring natural disasters - floods, landslides, earthquakes, wildfires - where warnings often arrive too late, or don't reach the people most at risk, especially in areas with weak connectivity. A credible system needs to operate at scale, pulling data from multiple sources and multiple response agencies, not just one location at a time.

Who This Is For

Residents in at-risk areas, local government/emergency officials, disaster relief agencies, and emergency response teams. You may choose any disaster type and region as your working context - just state your assumption clearly.

The Problem

Build a cloud-based system that ingests risk data at scale (weather feeds, sensor networks, satellite data, crowd-sourced reports), determines when risk is rising in specific areas, and gets targeted alerts to the right people fast - without alert fatigue for people outside the affected area.

Your Solution Needs to Handle

- Ingesting data continuously from many independent sources (sensors, weather APIs, citizen reports) at once
- A sudden, massive spike in activity across multiple regions simultaneously during an actual event
- Targeting alerts geographically - people outside the affected zone shouldn't be alerted
- Getting alerts to people with slow or unreliable internet (SMS fallback, offline-first considerations)
- The system staying fully operational during the disaster it's warning about, including partial connectivity loss
- Multiple agencies needing different views/access to the same underlying data

Special Considerations

This is safety-critical infrastructure. If your system goes down exactly when it's needed most, that's a real-world failure - explain your resilience approach specifically, including how you'd handle a regional outage without losing the whole system.

2. Scenario 2 - Photo & Video Social Platform

Background

A social platform (think Instagram-scale) where users share photos, short videos, and stories, follow each other, and interact through likes/comments/DMs. This is a full consumer social product aimed at global scale, not a small feature.

Who This Is For

A broad consumer user base (launch audience in the hundreds of thousands, with a path to scaling further), content creators within that community, and the platform's own moderation/ops team.

The Problem

Design the cloud architecture for a media-heavy social platform: users upload photos/videos, get an algorithmically ranked feed, follow/interact with others, and the platform stays fast and available at scale.

Your Solution Needs to Handle

- Massive media upload volume - storage, processing (e.g., resizing/transcoding), and fast delivery worldwide
- Feed generation that's personalized per user and stays fast even as users and posts grow
- Real-time-ish interactions: likes, comments, notifications arriving promptly
- Read-heavy traffic that dwarfs write traffic (many more viewers than posters)
- Search and discovery (finding people, hashtags, or content) at scale
- Content moderation - some mechanism for catching harmful/abusive content, even at a high level
- Cost control - media storage and bandwidth are usually the biggest cost driver here; show you've thought about it

Special Considerations

You don't need a perfect feed-ranking ML model - a reasonable, explained approach is fine. What's being judged is whether your infrastructure choices make sense for a media-heavy, read-heavy, high-growth consumer app.

3. Scenario 3 - Short-Form Video & Live Streaming Platform

Background

A short-form video platform (think TikTok/YouTube Shorts) combined with live streaming (think Twitch), aimed at a global audience. Users upload short videos that get algorithmically distributed to a personalized feed, and creators can go live with real-time viewer chat.

Who This Is For

General content viewers, content creators, and live streamers - potentially hundreds of thousands of concurrent users during peak/viral moments.

The Problem

Design the cloud architecture powering both the recorded short-video feed and the live-streaming side of the platform, including infrastructure for handling a video or stream suddenly going viral.

Your Solution Needs to Handle

- Video ingestion and processing (transcoding into multiple qualities/formats) at scale
- Fast global video delivery with low buffering
- Live streaming specifically - low-latency delivery plus real-time chat for potentially thousands of concurrent viewers on one stream
- A sudden, unpredictable spike in traffic to one specific piece of content without slowing the rest of the platform

- A personalized recommendation feed that serves fresh content quickly, not just a fixed replay list
- Storage lifecycle - not all video needs to stay on the most expensive storage tier forever

Special Considerations

Focus your depth on either the recorded-video path or the live-streaming path (or both, if your team can manage it) - say explicitly which you're prioritizing and why, rather than giving shallow treatment to everything.

4. Scenario 4 - Ride-Hailing & On-Demand Delivery Platform

Background

A ride-hailing and delivery platform (think Uber/Lyft/DoorDash) operating across multiple cities globally, matching riders/customers with drivers in real time, with live location tracking and in-app payments.

Who This Is For

Riders and delivery customers, drivers/couriers, and platform operations staff monitoring the system across multiple cities simultaneously.

The Problem

Design the cloud architecture for a real-time matching and logistics platform — from the moment a request comes in, to matching it with a nearby driver, to live tracking, to payment.

Your Solution Needs to Handle

- Real-time, high-frequency location updates from many drivers simultaneously (continuous GPS pings)

- Geospatial matching - finding the nearest available driver to a request quickly, at city scale
- Live tracking updates pushed to the rider/customer's app in near real time
- Surge/demand spikes at predictable times (e.g., rush hour, bad weather, big events) across multiple cities at once
- Payment processing, including handling failed/retried payments without double-charging
- The system continuing to operate correctly if one city's traffic spikes heavily while others stay normal

Special Considerations

Payment and location data are both sensitive - your proposal must explicitly address how each is protected, not just state "we use encryption." Also address what happens to an in-progress ride/delivery if a component fails mid-transaction.

5. Scenario 5 - AI Agent Platform for Business Automation

Background

Businesses increasingly want AI agents that can autonomously carry out multi-step tasks — answering customer questions, researching information, drafting documents, or triggering actions in other systems - rather than a single chatbot that just answers questions. This scenario is about the cloud infrastructure behind a platform that lets many businesses deploy and run their own AI agents at once.

Who This Is For

Businesses (the platform's customers) who configure and deploy agents, their end users interacting with those agents, and the platform's own team monitoring agent behavior across all tenants.

The Problem

Design the cloud architecture for a multi-tenant platform where each business can deploy one or more AI agents that use large language models, call external tools/APIs, retain some memory of past interactions, and take multi-step actions toward a goal - reliably, safely, and at reasonable cost.

Your Solution Needs to Handle

- Running LLM inference at scale for many businesses and end users simultaneously, with acceptable latency for an interactive experience
- Agent memory/context - retrieving relevant past information (e.g., via a knowledge base) without every request becoming enormous and expensive
- Tool-calling - agents safely calling external APIs or internal systems to actually take action, not just talk
- Guardrails - preventing an agent from taking a harmful, incorrect, or unintended action, and having a way to catch it when it does
- Observability - being able to see why an agent made a given decision, not just that it made one, for debugging and trust
- Cost control - LLM inference cost can spiral quickly; show you've thought about managing it (e.g., caching, model selection, rate limiting)
- Multi-tenant isolation - one business's data, agents, or usage spikes should not affect another's

Special Considerations

You are not expected to train your own model - assume you're calling existing LLM APIs/managed AI services. The focus is the platform and orchestration layer around the model, not the model itself. Given agents can take real actions, safety and auditability are scored seriously here, not treated as an afterthought.

Submission Requirements

- 1. Architecture diagram - see the separate Report & Diagram Template document for structure and formatting
- 2. Written proposal report (max 5 pages) - same template
- 3. Optional: a rough prototype or repo link - not required, and not scored separately if missing

What We Are NOT Looking For

- A copy of a tutorial architecture with your scenario's name swapped in
- Every possible cloud service crammed in "just in case" - pick what the problem actually needs
- Generic security/compliance language with no specifics
- Shallow, one-line treatment of every feature instead of real depth on the parts you chose to focus on

Judging

See the separate Evaluation Criteria document for the full scoring rubric.